

1. Heuristic design and admissibility

Let $D(n)$ be the set of positions where state n differs from goal g , and $r(w)$ the supplied rarity. All formulas return zero when $n = g$.

Steps: $h(n) = |D(n)|$. Every differing position must be replaced at least once. One action changes only one position and costs one, so any solution costs at least $|D(n)|$.

Scrabble: $h(n) = \text{sum over } i \text{ in } D(n) \text{ of } \text{value}(g[i])$. At each mismatched position, the goal letter must eventually be inserted at that position. These are distinct actions and incur the indicated replacement-letter costs. Extra actions only add positive cost.

Frequency: $h(n) = |D(n)| + r(g)$ when $n \neq g$, and zero otherwise. At least $|D(n)|$ actions are required. Every successful path from a non-goal state must enter g once, paying $r(g)$, while all other rarities are nonnegative. This remains admissible even if a path temporarily matches and then changes goal letters. Each heuristic is consistent: an action can remove at most one mismatch; Scrabble pays for any newly corrected goal letter; frequency pays the goal rarity on the last action.

2. Search comparisons

Nodes expanded are the values printed by the supplied search.py with `display=True`. Each run used a fresh DC instance. $\text{Reduction} = 100 \times (\text{UCS} - \text{A}^*) / \text{UCS}$.

Problem	Cost	UCS cost	UCS nodes	A* cost	A* nodes	Reduction
WHY → ASK	Steps	5	773	5	16	97.93%
WHY → ASK	Scrabble	9	765	9	13	98.30%
WHY → ASK	Frequency	38.8554	808	38.8554	634	21.53%
BEAR → DUCK	Steps	4	691	4	6	99.13%
BEAR → DUCK	Scrabble	11	2981	11	6	99.80%
BEAR → DUCK	Frequency	38.2402	1084	38.2402	302	72.14%

UCS and A* returned equal optimal costs in all six cases. Scrabble had the largest percentage reduction for each word pair; frequency had the smallest. The steps and Scrabble heuristics account for the unavoidable cost of correcting each position, whereas the frequency heuristic accounts only for one unit per correction plus the goal rarity. It cannot predict rarities of intermediate words, which can dominate the actual frequency cost. Search tie ordering also affects node counts.

3. Reversibility and direction-dependent cost

Steps: yes. Reversing a path preserves its number of actions. Scrabble: no. For one-step dictionary neighbors dog and cog, dog $\rightarrow$ cog inserts c (cost 3), whereas cog $\rightarrow$ dog inserts d (cost 2). Frequency: no. The same neighbors cost $1 + r(\text{cog})$ forward and $1 + r(\text{dog})$ backward. Their supplied rarities are $r(\text{cog}) = 11.393166$ and $r(\text{dog}) = 5.819885$. Since all edge costs are at least one, the direct edge is optimal for steps, but for frequency a cheaper indirect route could theoretically intervene; the measured reverse shortest-path costs below provide a verified frequency counterexample.

For the frequency counterexample, ASK $\rightarrow$ WHY has optimal cost 37.76 (rounded), while WHY $\rightarrow$ ASK has optimal cost 38.86 (rounded), computed with UCS/A*. Both are legal dictionary words. Actions are reversible, but entering the destination word determines the rarity charge, so reversing a route changes its edge costs.

4. Generative AI heuristic audit

System: GPT-6 Exact prompt:

“In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.”

Proposed heuristic: $h_1(n) = h_0(n) + r(g)$ for $n \neq g$; $h_1(g) = 0$. This is the rarity-aware heuristic implemented in dc.py. Every successful path from $n \neq g$ enters the goal and incurs $r(g)$; at least $h_0(n)$ actions each cost at least one. Thus h_1 cannot overestimate. It dominates h_0 everywhere and is strictly greater wherever $r(g) > 0$ and $n \neq g$. Both test goals satisfy this condition. No repair was required.

Frequency case	h0 cost	h0 nodes	h1 cost	h1 nodes
WHY $\rightarrow$ ASK	38.8554	755	38.8554	634
BEAR $\rightarrow$ DUCK	38.2402	720	38.2402	302

The verified heuristic preserved optimal costs and reduced expansions on both cases. Its proof establishes admissibility generally; these experiments support using it for these problems but do not guarantee a reduction for every input.

AI disclosure: OpenAI Codex (GPT-6) assisted with implementation and heuristic design. I checked argument handling and sample cases with dcsolve.py and dctest.py, compared UCS and A* costs on all required runs, and verified the heuristic with the proofs above. Source: supplied CMSC 471 assignment, dictionary, and AIMA search files.